

多 Agent 旅遊規劃系統

系統介紹、架構設計與技術演進報告

文件版本：	v3.0 — 2026-05-26
撰寫對象：	專案組員（簡報製作 / 架構理解）
系統狀態：	開發中（核心功能已驗證）
主要技術：	Azure OpenAI Google APIs Python / Streamlit

本報告涵蓋：

- 系統定位與使用者價值
- 核心特色功能詳解（含 AI 地點驗證）
- 多 Agent 架構演進歷程
- 各元件職責與 API 整合
- 端對端工作流程（Phase 0-3）
- 技術亮點與資料完整性設計

0 系統產品介紹

0.1 這個系統是什麼？

「多 Agent 旅遊規劃系統」是一套以 AI 驅動的全自動旅遊行程規劃平台，使用者只需輸入目的地、天數、預算與旅遊偏好，系統便能自動產生完整的逐日行程——包含景點、餐廳、移動路線、費用估算，並生成可下載的 PDF 旅遊手冊。

0.2 解決什麼問題？

- [x] 自行規劃行程耗時費力：蒐集資訊、排序景點、計算交通時間，往往需要數小時
- [x] LLM 幻覺風險：AI 直接建議的地名可能不存在或已歇業，無法信任
- [x] 預算難以掌握：景點票價、餐廳費用、交通成本分散各處，不易彙整換算
- [x] 行程不符實際節奏：純 AI 排程常忽略移動時間，導致每天行程過滿

0.3 我們的解法

- 一鍵生成：對話式輸入後，多個 AI Agent 協同工作，30 秒內產出完整行程草稿
- API 地點驗證：每個 LLM 提出的地點皆透過 Google Places API 確認真實存在，杜絕幻覺
- 真實路線計算：Google Routes API 計算實際交通時間（步行 / 大眾運輸 / 開車）
- 節奏智能管控：系統自動偵測移動時間過長，提示瓶頸路段並詢問使用者如何調整
- 即時預算換算：ExchangeRate-API 取得當日匯率，一鍵換算成台幣總費用
- 互動式地圖：路線以 Google Maps 彩色標記呈現，每天路線可單獨切換檢視
- 一鍵 PDF 輸出：含時間表、費用、備忘欄的正式旅遊手冊

0.4 目標使用者

自助旅遊者，特別是計劃前往日本（東京、大阪、京都）等地的台灣旅客；未來架構支援多城市串接，可擴展至任意國際目的地。

1 核心特色功能

[1] AI 地點驗證 (Place Grounding) —— 第一道防幻覺機制

問題背景：大型語言模型 (LLM) 有時會「想像」不存在的景點或餐廳名稱，
若直接使用，使用者抵達當地才發現地點不存在，體驗極差。

解決方案：每當 AI Agent 提出一個景點 / 餐廳名稱，系統立即呼叫

Google Places API (New) — searchText 端點，

確認該地名能對應到一個真實的 place_id。

驗證流程：

輸入：AI 建議的地點名稱 (如「大阪城公園」)

呼叫：Places API searchText，取得候選地點清單

輸出：PlaceStop(name, place_id) — 包含官方名稱與唯一 ID

失敗：若無結果，拋出 GroundingNotFound，觸發 Agent 重新提案

效果：100% 進入行程的景點皆有 place_id 背書，

後續 Google Routes API 計算路線時直接使用 place_id，精確無誤。

[2] 真實交通路線計算 (Google Routes API)

功能：計算景點間的實際交通時間與距離，支援大眾運輸 / 步行 / 開車三種模式。

計算邏輯：

每日路線 = 飯店 -> 景點1 -> 景點2 -> ... -> 飯店 (逐段計算)

優先嘗試大眾運輸；若 API 無結果，自動 fallback 至開車估算

雙距離指標 (新功能)：

- 總距離：所有交通模式的移動距離總和 (評估行程密度)

- 步行距離：僅計算 WALK 步驟 (評估體力負荷)

顯示格式：總距離 25.4 km (步行 3.4 km)

節奏管控整合：若總移動時間超過使用者設定的步調上限，

自動標示最耗時的瓶頸路段 (原點→終點，耗時 X 分鐘)。

[3] 即時匯率換算與預算管控 (ExchangeRate-API)

功能：在行程確認時自動取得當日即時匯率，將所有費用統一換算為台幣顯示。

費用來源 (多層次收集)：

- 飯店費用：使用者輸入 + LLM 查詢

- 景點票價：Google Places API price_level + LLM 估算

- 餐廳費用：LLM 根據類型估算

- 交通費：Google Routes API transitFare (實際票價)

預算警示：每次新增景點後即時計算累計費用；

若超出使用者預算上限，觸發 BUDGET_EXCEEDED 衝突事件。

PDF 輸出：費用概覽頁含匯率快照 + 已確認費用總計。

[4] 互動式路線地圖 (Google Maps + Encoded Polyline)

兩種地圖模式：

- (A) 預覽地圖：規劃初期顯示飯店候選位置與必訪景點，協助使用者選擇飯店
- (B) 驗證路線地圖：每天行程確認後，顯示實際路線

路線地圖特色：

- Google Routes API 回傳 encoded polyline，直接渲染於地圖
- 標記顏色依類型區分：景點 (藍) / 午餐 (橘) / 晚餐 (紅) / 飯店 (綠)
- 側欄日期切換：可單獨檢視每天的行動路線
- 標記編號對應時刻表順序，便於對照行程

[5] 一鍵輸出專業旅遊手冊 (PDF)

使用 fpdf2 產生含 CJK 字型 (Microsoft JhengHei) 的 PDF，支援繁體中文。

PDF 內容：

- 封面摘要：目的地、天數、預算、步調、興趣標籤
- 費用概覽：匯率快照 + 已確認費用總計
- 每日時刻表：時間 / 類型 / 地點 / 備註 (自動換行表格)
- 路線摘要：移動時間 / 最長單段 / 總距離 / 步行距離
- 旅行備註頁：空白格線 (手寫用)
- 緊急聯絡資訊：含保單、駐外館處、信用卡掛失等填寫欄

2 架構演進：從單一 LLM 到多 Agent 協作

本系統經過三個主要世代的設計演進，逐步從「提示工程」升級至「多 Agent 協作 + 真實 API 驗證」的架構典範。

面向	第一代（原型）	第二代（單 Agent）	第三代（現行）
規劃主體	單一 ChatGPT 提示	單一 Agent + 工具	5 個專職 Agent 協作
地點可靠性	LLM 自由想像	LLM 自由想像	Google Places API 驗證
路線計算	LLM 估算（不準確）	靜態距離估算	Google Routes API 實算
預算追蹤	無	使用者手動核算	自動累計 + 匯率換算
衝突解決	無	重新生成整份行程	事件驅動式決策閘道
使用者互動	一次性輸出	有限對話	逐日決策 + 地圖 + PDF

核心典範轉移：從「生成後使用」到「生成前驗證」——

每一個 AI 提案在進入行程前，必須通過 Google API 的現實世界核驗。

3 系統元件：五大 AI Agent + 外部 API 層

3.1 AI Agent 層 (Azure OpenAI)

DestinationAgent | 目的地解析

職責：解析使用者輸入的目的地文字，確認城市名稱與座標基準
呼叫：Google Places API — lookup_destination()，確認目的地真實存在
輸出：標準化目的地名稱 + place_id

HotelAgent | 飯店篩選

職責：根據位置偏好、預算、旅遊類型推薦飯店候選清單
呼叫：Google Places API — search_hotel_candidates()，取得最多 3 間真實飯店
互動：在地圖上顯示飯店位置，讓使用者選擇
輸出：Hotel(name, place_id, address, lat, lng)

AttractionAgent | 景點規劃

職責：為每一天規劃景點清單，考量使用者興趣、步調、節奏限制
驗證：每個景點名稱皆透過 Google Places API — ground() 驗證存在性
失敗處理：GroundingNotFound 時自動請 Agent 替換備選景點
輸出：PlaceStop(name, place_id) 清單

MealAgent | 餐廳規劃

職責：為每天的午餐 / 晚餐推薦符合預算與口味的餐廳
驗證：同樣透過 Google Places API — ground() 確認餐廳存在
輸出：PlaceStop(name, place_id) for LUNCH / DINNER 時段

BudgetAgent | 預算管控

職責：彙整所有費用項目，計算當日 / 全程預算使用情況
來源：景點票價 (Places API price_level)、餐廳估算、Routes API 票價
換算：ExchangeRate-API 當日匯率 → 台幣顯示
衝突：超出預算上限 → 觸發 BUDGET_EXCEEDED 事件，詢問使用者調整方式

3 系統元件 (續) : 外部 API 整合

3.2 系統工具層 (外部 API)

API 服務	端點 / 模組	系統用途
Google Places API (New)	searchText	地點驗證 (Grounding) : 確認景點 / 餐廳 / 飯店真實存在
Google Places API (New)	searchNearby	飯店候選 : 依距離搜尋周邊住宿
Google Routes API	computeRoutes (TRANSIT)	逐日路線 : 大眾運輸時間 / 票價 / polyline
Google Routes API	computeRoutes (DRIVE)	Fallback : 無大眾運輸時改用開車估算
ExchangeRate-API	latest/{base}	匯率快照 : JPY -> TWD 即時匯率
Azure OpenAI	chat/completions	所有 5 個 Agent 的 LLM 推理後端

3.3 資料模型 (核心 Domain Model)

所有元件間的資料傳遞均透過 Pydantic v2 模型，確保型別安全與 JSON 序列化一致性。

TripSpec	完整旅程規格：目的地、天數、預算、步調、飯店、興趣標籤
DayPlanState	單日行程狀態：景點 / 餐廳清單、時刻表、路線結果、費用
PlaceStop	已驗證地點：name + place_id (由 Places API 核發)
RouteEvidence	路線證據：移動時間、步行距離、總距離、polyline 路段
ConflictEvent	衝突事件：PACE_EXCEEDED / BUDGET_EXCEEDED + 瓶頸分析
PricItem	費用項目：金額、幣別、狀態 (已確認 / 估算 / 缺失)
FxSnapshot	匯率快照：base_currency、target_currency、rate、擷取時間

4 端對端工作流程 (Phase 0 → 3)

Orchestrator 統籌協調全部 Agent · 依序執行四個階段。每個階段均有明確的輸入 / 輸出契約，失敗時有對應的錯誤處理路徑。

Phase 0 | 旅程初始化 (使用者輸入)

- 1. 使用者輸入：目的地 / 天數 / 預算 / 步調 / 興趣
- 2. DestinationAgent：Places API 驗證目的地，取得 lat/Ing
- 3. HotelAgent：Places API 取 3 間候選飯店 → 地圖顯示 → 使用者選擇
- 4. ExchangeRateClient：取得當日即時匯率快照
- 5. 輸出：TripSpec (含飯店 place_id + 匯率)

Phase 1 | 景點生成 (每天重複)

- 1. AttractionAgent：依步調產生景點清單 (LLM 提案)
- 2. Places API ground()：逐一驗證每個景點名稱 → PlaceStop(name, place_id)
- 3. GroundingNotFound：自動替換備選景點 (無人工介入)
- 4. Routes API computeRoutes()：計算「飯店→景點→飯店」初始路線
- 5. 節奏評估：若移動時間超標，觸發 PACE_EXCEEDED 衝突

Phase 2 | 餐廳安排 + 決策閘道

- 1. MealAgent：依餐廳類型偏好提案午餐 / 晚餐
- 2. Places API ground()：驗證餐廳真實存在
- 3. Routes API：重新計算含餐廳的完整日路線
- 4. BudgetAgent：統計當日費用，確認不超出預算
- 5. 決策閘道 (Decision Gate)：呈現完整日程 + 地圖 → 使用者批准 / 調整 / 捨棄

Phase 3 | 行程輸出

- 1. 所有天次批准後，彙整 DayPlanState 清單
- 2. 生成逐日時刻表 (含時間 / 地點 / 交通備註)
- 3. PDF 生成：generate_itinerary_pdf() → 含時刻表 + 地圖 + 費用 + 備忘欄
- 4. 下載按鈕：st.download_button() 提供 PDF 二進位下載

5 衝突事件系統 (Event-Driven Conflict Resolution)

當行程違反使用者限制時，系統不會直接失敗，而是觸發

ConflictEvent，透過決策問道詢問使用者如何處理。這保留了使用者的主控權，同時避免 AI 自作主張修改行程。

PACE_EXCEEDED | 行程節奏超標

觸發條件：單日移動時間超過使用者設定的步調上限 (如標準步調 = 120 分鐘 / 天)

瓶頸分析：系統找出移動時間最長的路段，顯示「從 A 到 B 需耗時約 X 分鐘」

使用者選項：(a) 保留行程 (b) 請 AI 移除一個景點 (c) 手動指定移除

BUDGET_EXCEEDED | 預算超出上限

觸發條件：累計已確認費用 (含票價 + 餐廳估算 + 飯店) 超過預算上限

顯示資訊：已確認費用總計 vs. 預算上限，以台幣呈現

使用者選項：(a) 擴大預算 (b) 請 AI 建議替代景點 (c) 繼續規劃

6 技術亮點與工程決策

雙距離指標 (新設計)

過去只計算步行距離 (WALK 步驟)，導致「移動 141 分鐘 但距離只有 3.4 km」的矛盾數據

現在同時追蹤 total_distance_km (所有交通模式) 與 walking_distance_km (僅步行)

顯示格式：「總距離 25.4 km (步行 3.4 km)」，清楚表達搭車距離與步行負擔

Session State 向後相容機制

問題：Streamlit session_state 儲存的 Pydantic 物件實例，在模型欄位新增後不會自動更新

解法：應用啟動時偵測 RouteEvidence.model_fields 是否包含新欄位；

若無，清除 session_state 中的舊物件並重新初始化

防禦層：所有 UI 顯示點加上 getattr(route, 'total_distance_km', 0.0) 保護

多城市 GlobalMemory

目的：跨城市旅遊時，Agent 需記住前幾天的行程以避免重複景點

設計：GlobalMemory 物件隨 Orchestrator 傳遞，各 Agent 讀寫共享上下文

效果：Day 1 去過「金閣寺」，Day 3 規劃京都時 Agent 自動排除

步調等級系統 (PaceProfile)

四個步調：非常悠閒 / 悠閒 / 一般 / 密集，對應不同的景點數 / 移動時間上限

PaceProfile 包含：max_major_places_per_day (主景點數)

max_required_transfer_minutes_per_day (每日移動時間上限)

max_single_transfer_minutes (單段移動上限)

walking_distance_warning_km (步行警示距離)

7 總結與系統現況

7.1 已完成功能

- [v] 多 Agent 協作架構 (DestinationAgent / HotelAgent / AttractionAgent / MealAgent / BudgetAgent)
- [v] Google Places API (New) 地點驗證 (Place Grounding) — 防幻覺第一層
- [v] Google Routes API 真實路線計算 (大眾運輸 + 開車 fallback)
- [v] 雙距離指標 : total_distance_km + walking_distance_km
- [v] 即時匯率換算 (ExchangeRate-API)
- [v] 衝突事件系統 : PACE_EXCEEDED (含瓶頸路段分析) 、 BUDGET_EXCEEDED
- [v] 互動式地圖 (預覽地圖 + 驗證路線地圖 · 含 encoded polyline)
- [v] 逐日時刻表生成 (含時間 / 地點 / 備註)
- [v] PDF 旅遊手冊輸出 (fpdf2 + CJK 字型 · 含費用 / 地圖摘要)
- [v] Session State 向後相容機制
- [v] 多城市串接基礎 (GlobalMemory)
- [v] 步調等級系統 (PaceProfile · 四種模式)

7.2 技術棧摘要

語言 / 框架 :	Python 3.12 Streamlit Pydantic v2
LLM 後端 :	Azure OpenAI (GPT-4o)
地圖 / 路線 :	Google Places API (New) Google Routes API
匯率 :	ExchangeRate-API (openexchangerates.org)
PDF 產生 :	fpdf2 2.8+ Microsoft JhengHei (CJK 字型)
測試 :	pytest + pytest-httpx (HTTP mock)
可觀測性 :	Langfuse (LLM call tracing)
套件管理 :	uv (pyproject.toml)

本系統的核心主張 : AI 生成行程 · API 驗證現實。

每一個進入行程的地點 · 都有 Google place_id 作為現實世界的錨點。